PROJECT REPORT OF INDUSTRY ORIENTED HANDS ON EXPERIENCE (IOHE)

ON

Unisphere – University Club & Event Management Platform

submitted in partial fulfilment of the requirements for the award of degree of

BACHELOR OF ENGINEERING

In

COMPUTER SCIENCE AND ENGINEERING

Submitted by:

Rohit Dogra

2211985041

Supervised By:

Dr. Rajat Takkar

Assistant Professor

Chitkara University

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY
CHITKARA UNIVERSITY, PUNJAB, INDIA

CONTENTS

List of Figures

DECLARATION

I hereby certify that the work which is being presented in the project report entitled **"Unisphere – University Club & Event Management Platform"** in partial fulfilment of requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of my own work carried out under the supervision of Dr. Rajat Takkar (Mentor). The matter presented in this project report has not been submitted in any other university/institute for the award of any degree.

Place: Rajpura **Rohit Dogra**

Date: 15-05-2026 University Roll No. - 2211985041

This is to certify that the above statement made by the candidate is correct to the best of my knowledge and belief.

Dr. Rajat Takkar
Assistant Professor
Department of Computer Science and Engineering
Chitkara University Institute of Engineering and Technology,
Chitkara University, Punjab, India

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my respected mentor Dr. Rajat Takkar for his continuous guidance, encouragement, and valuable suggestions throughout the development of the project **"Unisphere – University Club & Event Management Platform"**. His technical expertise and mentorship played a significant role in shaping the direction of this project.

I am also thankful to the Department of Computer Science and Engineering, Chitkara University, for providing the infrastructure, resources, and learning environment necessary for successful project development.

Finally, I express my gratitude to my friends and family members for their constant motivation and support throughout the project lifecycle. Their encouragement kept me focused during the more challenging phases of development.

Abstract

There are always numerous events happening at universities, along with a large number of clubs and educational resources. Yet, due to poor communication between WhatsApp groups, emails, and notice boards, students are not aware of these opportunities. Consequently, this results in low attendance, poor event management, and an inability to hold club members accountable within the university structure.

Unisphere is a digital platform designed to manage events and clubs within universities. It provides a space for students to search for clubs according to their preferences, sign up for events, and communicate with other students in real time. Every event is managed by its own approval life cycle consisting of six stages from drafting to completing. Hence, this guarantees faculty oversight before the event becomes accessible to students. User registration is controlled by administrators based on the CSV file upload.

The project was developed using the MERN stack—React.js, Node.js, Express.js, and MongoDB Atlas. The application uses JSON Web Tokens (JWT) for role-based authentication, Socket.IO for real-time communication, and BullMQ connected with Redis for background tasks processing.

Unisphere is expected to improve connections on campus, increase engagement among students, and limit their dependence on non-official sources.

1. Introduction

In most universities, the information regarding various clubs, events, seminars, and other activities are circulated via unstructured ways such as WhatsApp groups, emails, and notice boards. Such distribution system leaves students—especially freshers—lacking any sort of formal medium to access and participate in university activities. By the time a student knows about the happening of such an activity, they are already over. Due to the lack of proper structure, students often end up missing the clubs of their liking and events concerning their academic/extracurricular purposes.

Similar issues arise in case of faculty members, too. If one professor is managing several clubs, then he/she would have to manage several different means of communications, which adds up to a lot of effort to maintain. There is no formal way to help professors keep track of their club activities and events taking place under the university's jurisdiction, as well as ensure the institution’s involvement and hold accountable for such events. Hence, campus life remains poorly engaged, poorly coordinated, and lacks any proper record keeping of decision-making process.

Unisphere came to solve all these problems. It is a web platform aimed at solving the problem of club and event management in universities. It consolidates club discovering, event organizing, and community discussion in one structured online platform. The idea behind developing this platform is that university club and event management problems arise not due to lack of interest but because of the lack of proper infrastructure. And that is what Unisphere seeks to bring to university students. It replaces WhatsApp groups, printed fliers, and unorganized communications in general with a structured, controlled and interest-focused platform that caters to three types of users—students seeking campus life engagement, faculty advisors managing institutional responsibilities, and administrators managing system operation.

The use of the platform is restricted to certain parties, and the website is not accessible to public in any manner. An admin needs to upload a CSV file containing user information of both students and faculty advisors, and the system generates accounts for users, sending out emails to users in the file. Post the authentication stage, users’ rights depend on their roles in the platform. During the registration process, students pick some interest tags, and according to

them, they are presented with suitable clubs. Students can participate in discussion forums, sign up for events, send direct messages to each other, and even receive notifications. Events are reviewed by faculty advisors using a special dashboard before they become visible to students.

The development process involves MERN stack, with React.js, Node.js, and Express.js for the front-end and back-end and MongoDB Atlas as the database. For secure role-based authentication, JWT is used, while for real-time communication, Socket.io is utilized, and for background job processing, which includes automatic deletion of events groups after completion of events, BullMQ with Redis is used.

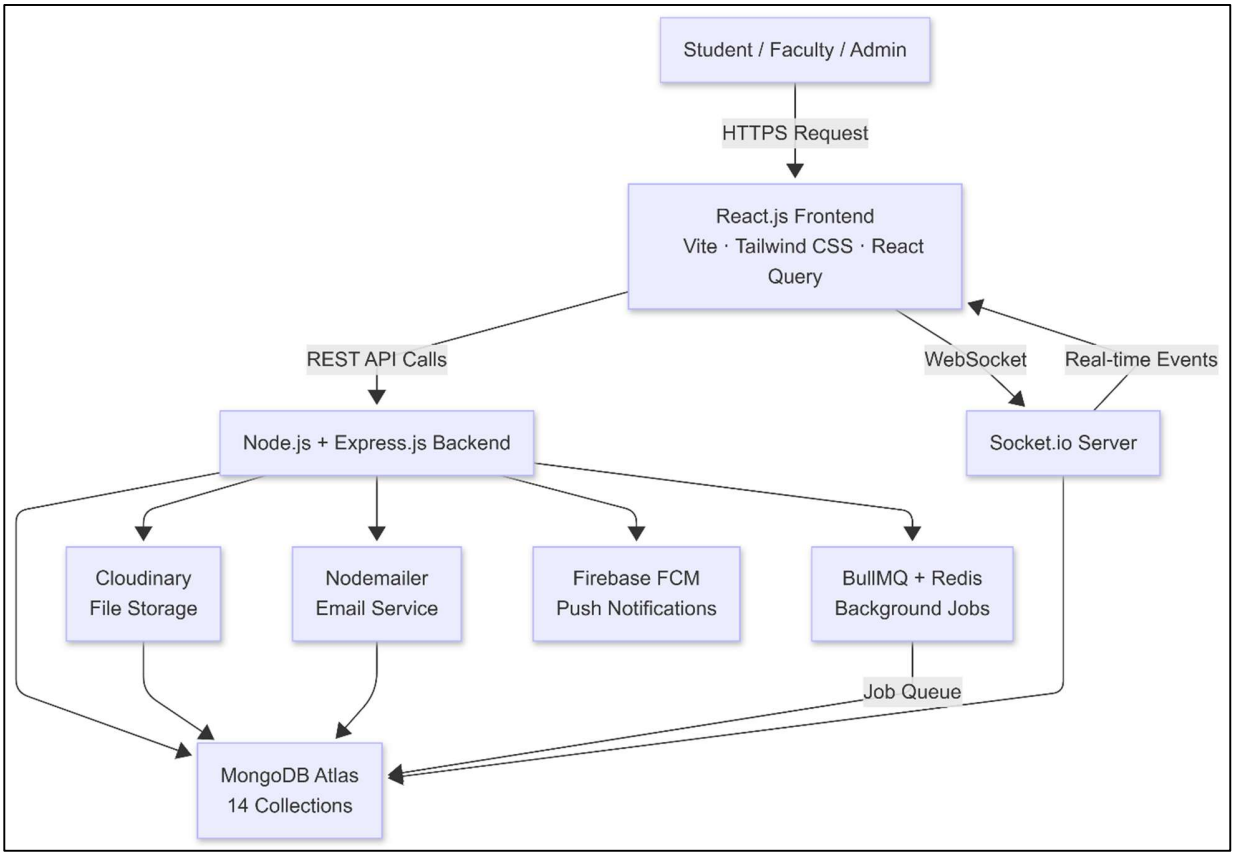
The rest of this report is structured as follows. The methodology and the system architecture are described in section 2. Section 3 describes UML diagrams and consists of class and use case diagrams. User Interface Design will be discussed in section 4. The tools and technology used will be presented in section 5. Implementation process will be explained in section 6. Results and Outcomes will be discussed in section 7. Conclusions will follow in section 8.

2. Methodology

2.1. Development Approach

Unisphere has been designed following the methodology of iterative and incremental development, whereby the system is constructed and verified independently in different functional modules instead of being implemented in a single go. Initially, the project focused on developing authentication, role-based access control, and user signup, then club administration, event posting and approval process, instant messaging, notifications, and eventually the administrator's console. This strategy helped test each component separately, find bugs, and make sure that the basic functions worked well before adding other functionalities.

2.2. System Architecture



The Unisphere platform uses a three-layer client-server architecture that involves a frontend layer, a backend layer, and a database layer.

- **Frontend Layer** – Built using React.js framework and Vite as the build tool. Manages the user interface, routing on the client side, and dashboard views for students, instructors, and administrators. The Tailwind CSS framework manages responsive design and Framer Motion handles animation and transitions between pages.
- **Backend Layer** – Developed using Node.js and Express.js frameworks. Provides RESTful API services for executing the business logic, user authentication, authorization, event handling, club administration, and communication. Socket.io is implemented on the same server to facilitate two-way communication.
- **Database Layer** – Built using MongoDB Atlas and Mongoose ORM. MongoDB was chosen because of its flexible document model that works best with data of various structures such as users, clubs, events, messages, and audits.

The background processes are carried out using BullMQ, while Redis is utilized for managing the job queues. The node-cron scheduler works at intervals of five minutes to look into the event timings and automatically change their statuses; where an event will get a status of "live" if its start time is met, while a live event gets a status of completed upon the end of its period, and hence dissolving of the group occurs.

2.3. Database Design

The database is organized into 14 collections in MongoDB Atlas:

Collection	Description
students	Stores student details: name, email, roll number, department, hashed password, interest tags, joined clubs, onboarding status, and active status
faculties	Stores faculty records: name, email, employee ID, department, role (faculty or hod), assigned clubs, active status
admins	Stores admin accounts: name, email, hashed password, role (admin or superadmin), active status

clubs	Stores club data: name, description, department, status (pending/active/inactive), president, vice president, faculty advisors, members, interest tags, profile picture URL
events	Stores event details: title, description, club reference, event type, status (draft/pending/approved/live/completed/cancelled), venue, timing, max participants, registration deadline, poster URL, organizers, event group reference
registrations	Links students to registered events: event ID, student ID, club ID, registration status (registered/attended/cancelled), check-in timestamp
eventgroups	Stores group chat rooms for events: event reference, club reference, name, status (active/dissolved), members with roles
messages	Stores all chat messages: room reference, room type (EventGroup/Club/DirectConversation), sender, message type (text), content, file details, soft delete fields
directconversations	Stores one-to-one private conversations: participants with user model references, last message timestamp, participant hash for deduplication
notifications	Stores user notifications: recipient, notification type, title, message, related entity reference, read status, read timestamp
notices	Stores official notices: posted by, role, title, content, club reference, attachment URL, expiry date, priority level, active status

approvallogs	Stores full audit trail: club reference, event reference, approver identity, action taken (submitted/approved/rejected/cancelled), written feedback, snapshot of event data before action
feedbacks	Stores post-event feedback: event reference, student reference, rating (1-5), comment text, anonymous flag
systemconfig	Stores platform configuration values: interest tag lists, event types, status codes, and other system-level settings managed by administrators

2.4. User Roles and Access Control

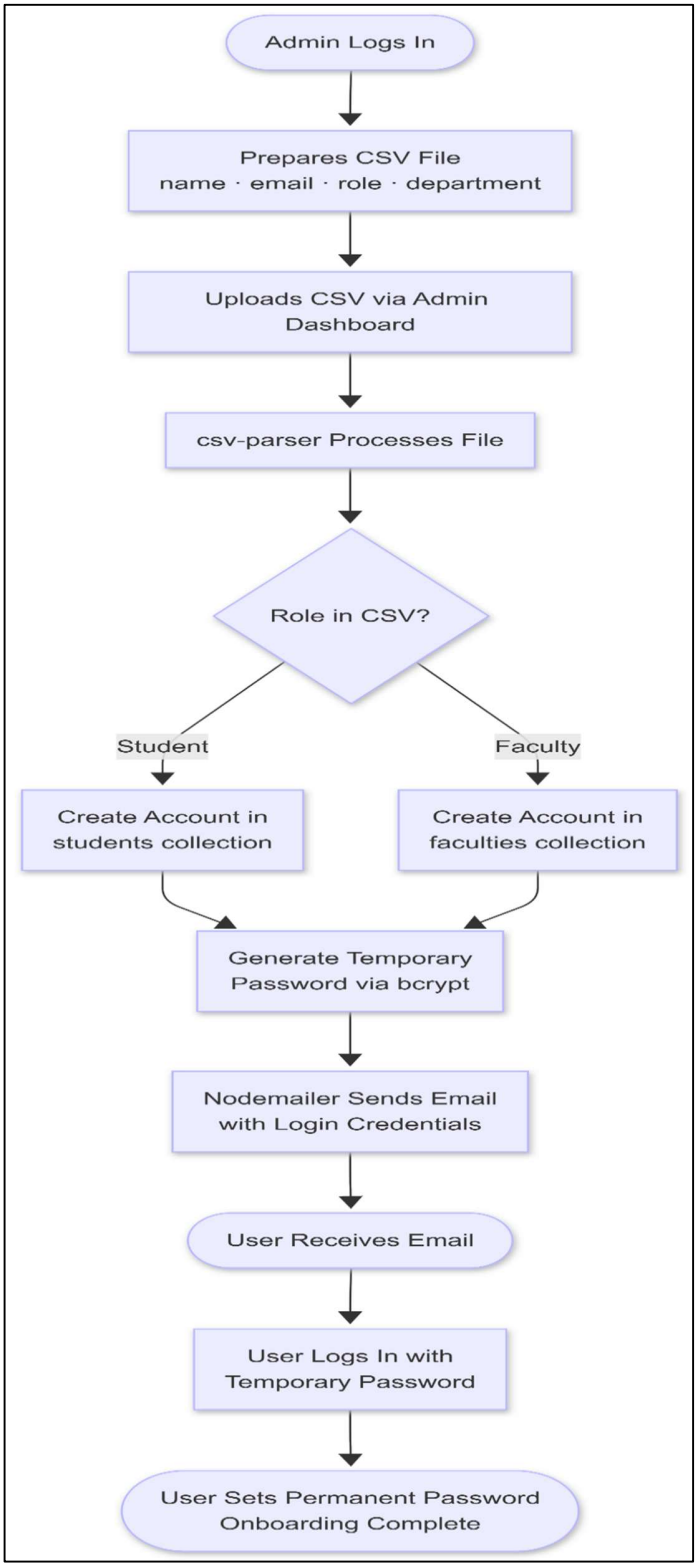
The Unisphere system uses the concept of role-based access control to control which features of the system can be used based on the user's level. The Unisphere system doesn't use an open-ended approach where users can sign themselves up. Instead, permission is granted via the onboarding procedure controlled by the institution.

Role	Responsibilities
Student	Uses interest tags to discover clubs, joins clubs, registers for permitted events, takes part in discussion boards and direct messaging, gets real-time notifications, sends feedback after events
Faculty	Serves as compulsory advisor for allocated clubs, reviews pending events, approves or declines events based on feedback, views club performance using analytics dashboard
Admin	Handles onboarding of users by uploading CSV files, approves formation of clubs, allocates faculty advisors

	to clubs, manages system activities using analytics dashboard
Superadmin	Is an administrative entity with enhanced system level access such as admin account management and platform-wide management using systemconfig collection

The mongoose schema validates data to match a specific format before saving it into the MongoDB database. Passwords are salted and hashed by bcrypt with a cost factor of 10. The JWT token carries the user's role and expires within a day for the access token and a week for the refresh token.

2.5. User Onboarding Flow

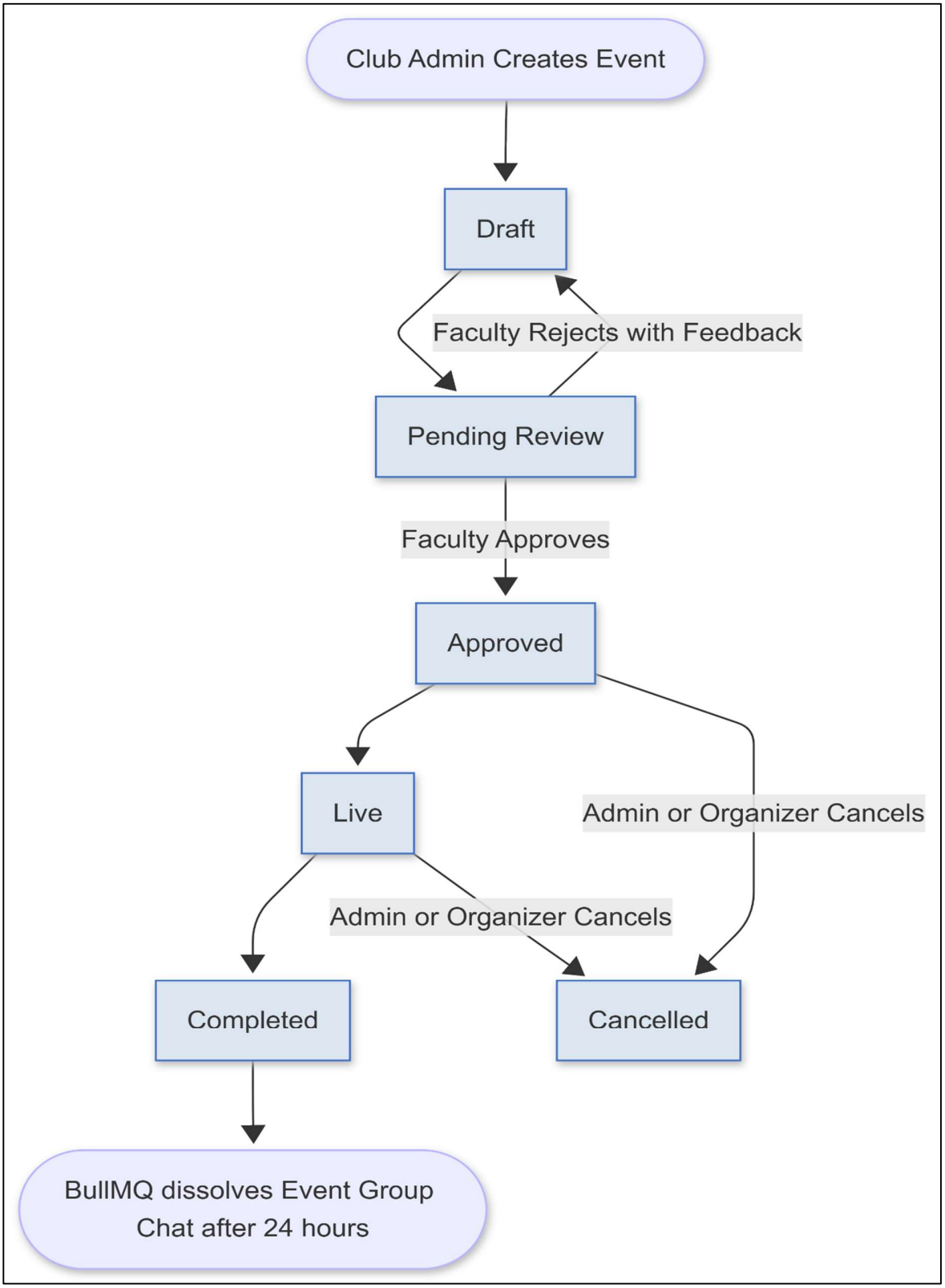


Unisphere makes use of administrator-controlled onboarding instead of making the system publicly available through open registration, thus guaranteeing access restriction to system users who have been verified as university affiliates. This process involves the following steps:

1. Creation of a CSV file containing details about the user, such as his/her name, email address, roll number or employee number, department, and role by the administrator.
2. Upload of the CSV file via the admin panel, where there is a feature for bulk import of the user data.
3. Automatic creation of user accounts based on the parsed information from the CSV file by csv-parser.
4. Automatic generation of passwords for the individual users.
5. Sending out of automated emails to each user registered on the system using Nodemailer with login information.
6. User logging into the system using the created login credentials and creating a permanent password after completing the profile information.

This allows for the elimination of having to create accounts manually while maintaining governance by the institution over accessing the platform. Only those records that have been included in the CSV upload will have access to the system.

2.6. Event Approval Workflow



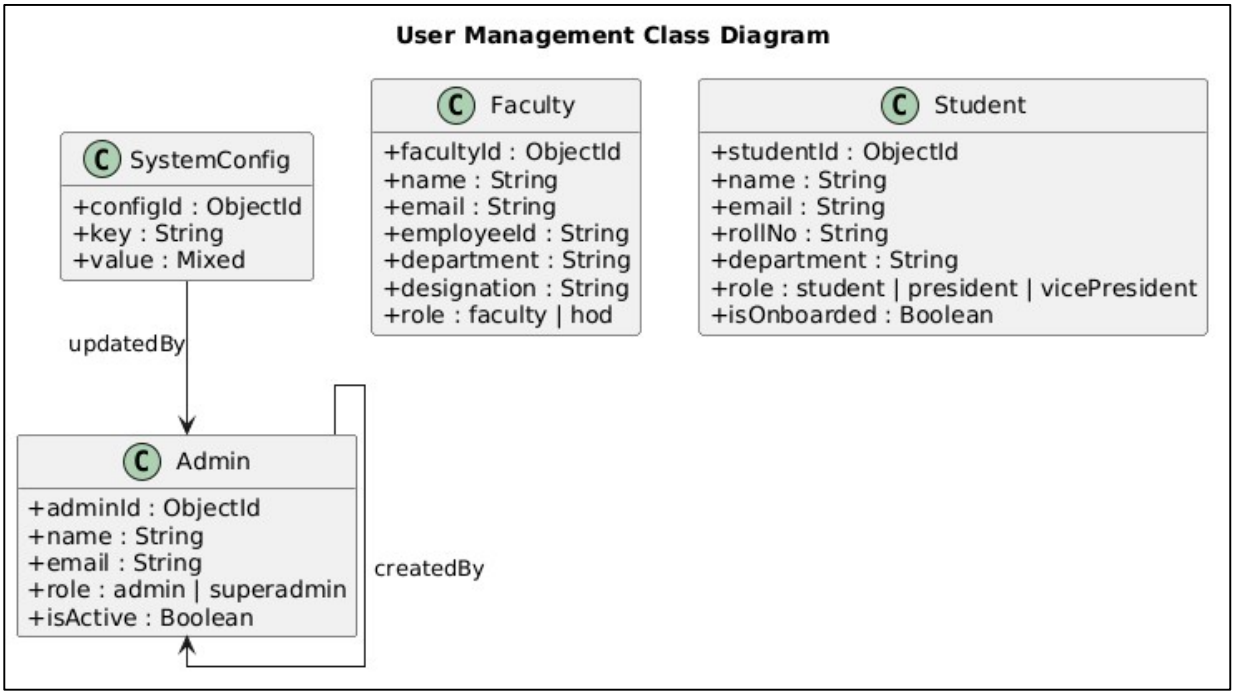
Each activity that occurs in Unisphere follows a tightly controlled six-step approval life cycle before being made available to students. This process guarantees faculty oversight and ensures that only verified activities can be published in the platform.

Stage	Description
Draft	The club administrator drafts the activity but does not publish it yet.
Pending Review	The club administrator publishes the activity pending faculty verification and review. It will show up in the faculty approval panel.
Approved	The faculty approves the activity, thus making it visible to students for them to register
Live	The start time of the activity has been reached. The status transitions automatically via node-cron at an interval of five minutes.
Completed	The end time of the activity has been reached. The status transitions automatically via the cron job. The event group chat associated with the activity is then terminated via BullMQ.
Cancelled	The organizer or club administrator cancels the activity, thus closing registration for the activity.

In case the event fails to pass verification due to rejection by the faculty, a justification must be provided by the faculty in form of written feedback, which is stored in the approval logs collection along with a complete copy of the event information when rejected.

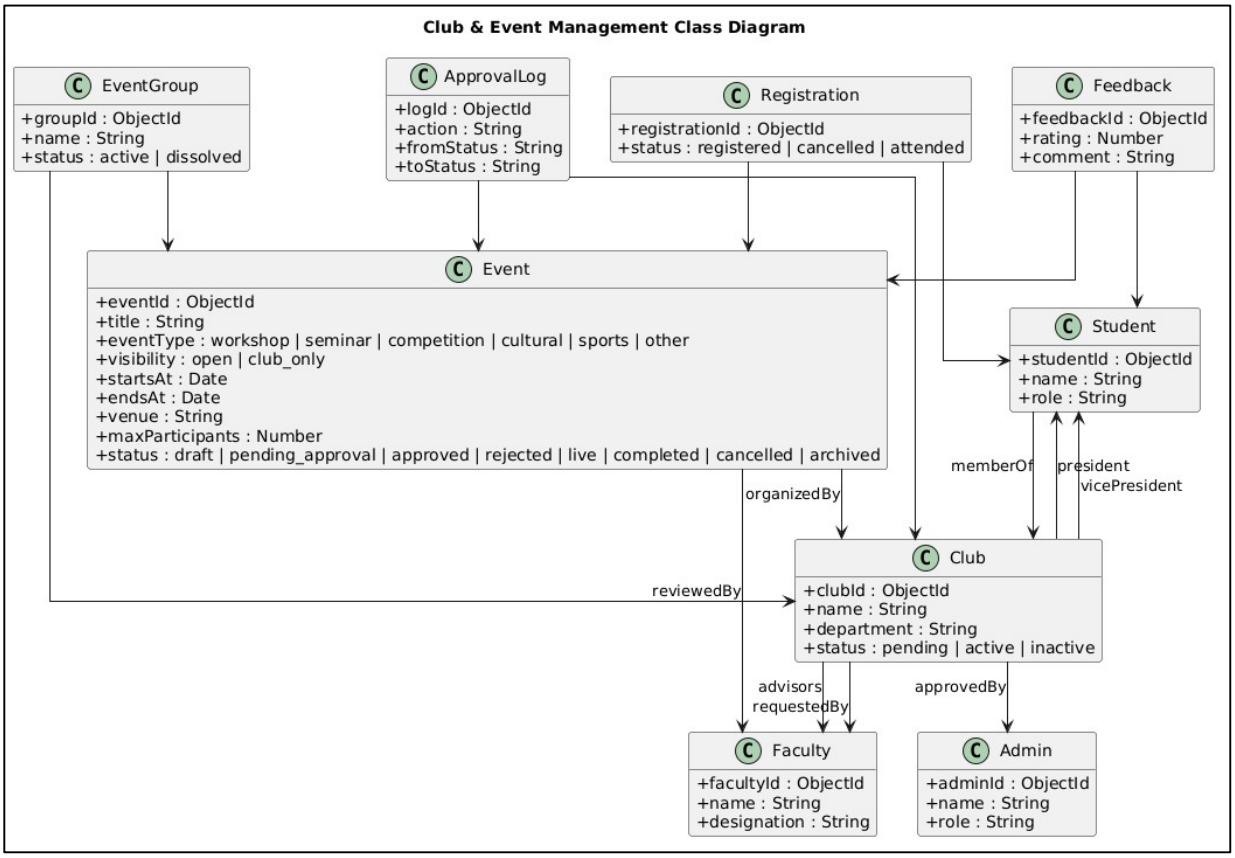
3. Diagrams

3.1. User Management Class Diagram



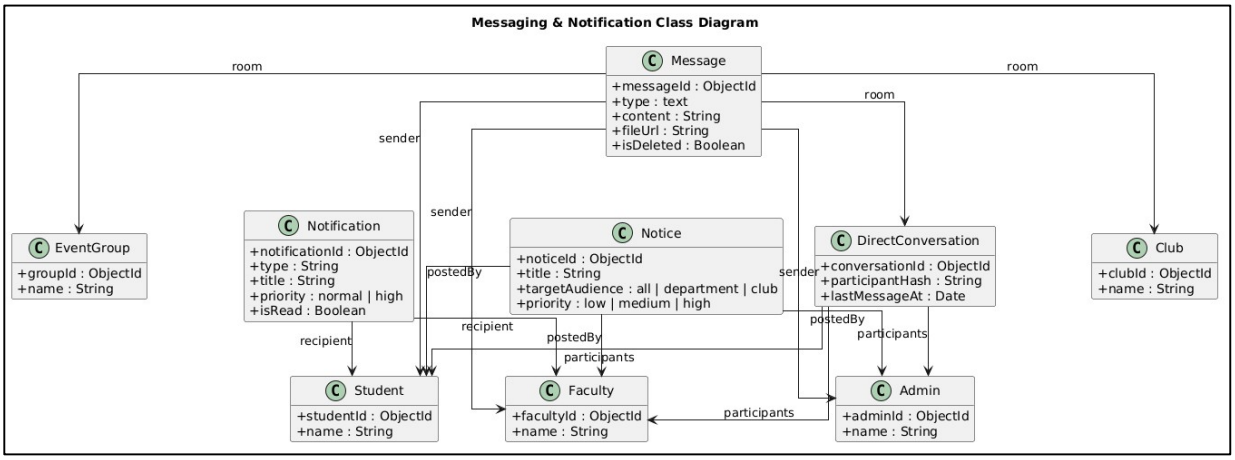
The User Management Class Diagram identifies the primary users in the Unisphere application, which include the Admin, Faculty, and Student. The Admin class is responsible for executing high-level processes such as user management, permission setting, and updating configurations in the application. The Faculty object includes information regarding the department, designation, and role of the faculty. The Student object manages the details of students and their onboarding process in the application. The SystemConfig class contains the system's configurable parameters and is controlled by the administrator.

3.2. Club & Event Management Class Diagram



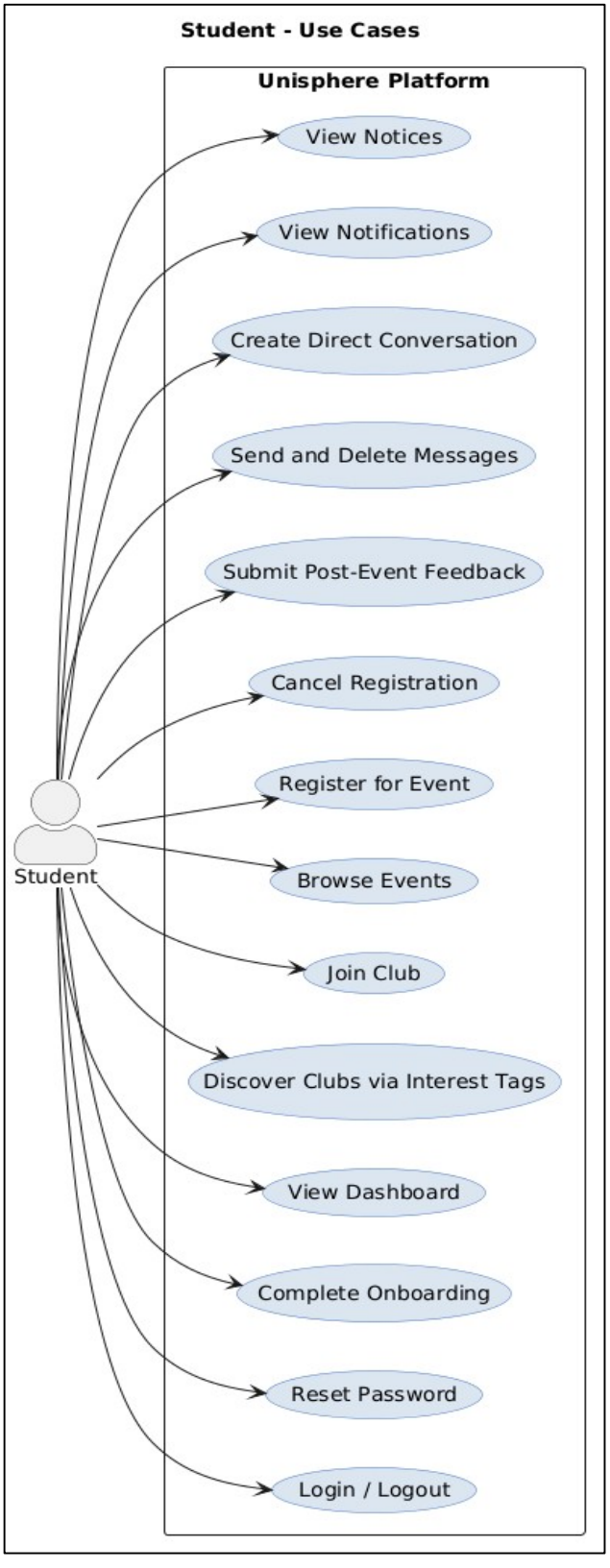
Class Diagram for Club & Event Management illustrates the hierarchy that controls club organization, event scheduling, registration, event review, and approval. Club represents the data about clubs, such as the advisor, president, and members of the club. Event manages the scheduling of events, their availability, the venue of the event, and the states of the event. EventGroup provides a channel through which participants communicate with each other during an event. Registration registers the attendance of students in an event, and Feedback logs the review provided by students. ApprovalLog stores the history of event approval actions taken and the state transitions.

3.3. Messaging & Notification Class Diagram



Messaging & Notification Class Diagram represents the communication module of Unisphere. Message is used for managing communication in the form of messages between members of clubs and events as well as in direct messaging. DirectConversation manages communication between two individuals including students, faculty members, and administrative personnel. Notice manages official communications made by organizations to various departments, clubs, or the whole system. Notification holds alerts generated by the system related to events, registrations, clubs, and messaging.

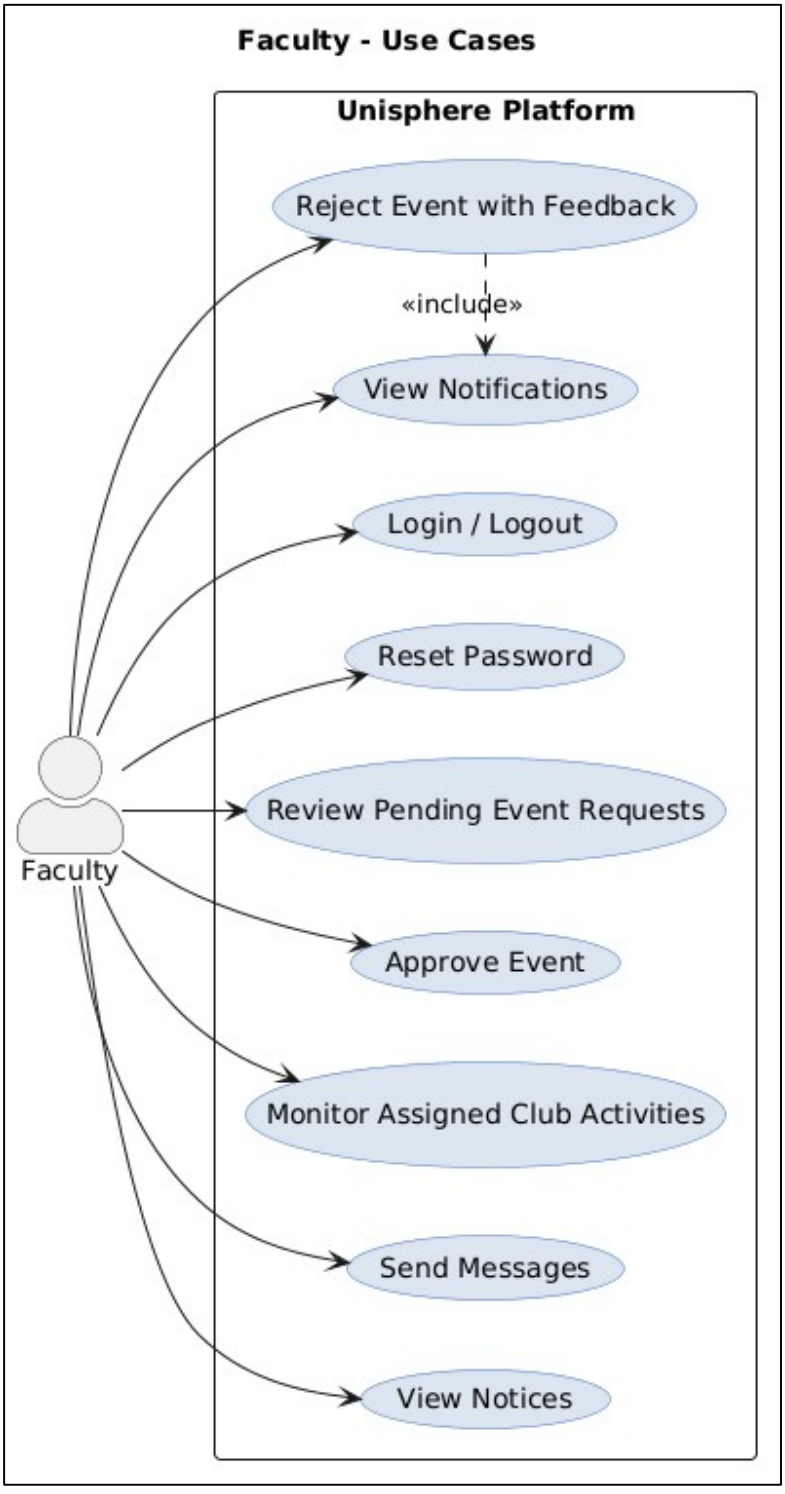
3.4. Student Use Case Diagram



The Student Use Case Diagram specifies the functionalities that students will have access to when using the Unisphere platform. These include logging in securely, completing their onboarding process, identifying clubs based on tags that reflect their interests, browsing future events, registering for the events approved by the system, joining clubs' communities, and using the instant messaging feature.

Moreover, students will be notified concerning event approvals, registrations, announcements, and club-related activities. Other functionalities will include holding individual conversations, receiving notices from lecturers/administrators, submitting feedback after events, and managing their event registrations.

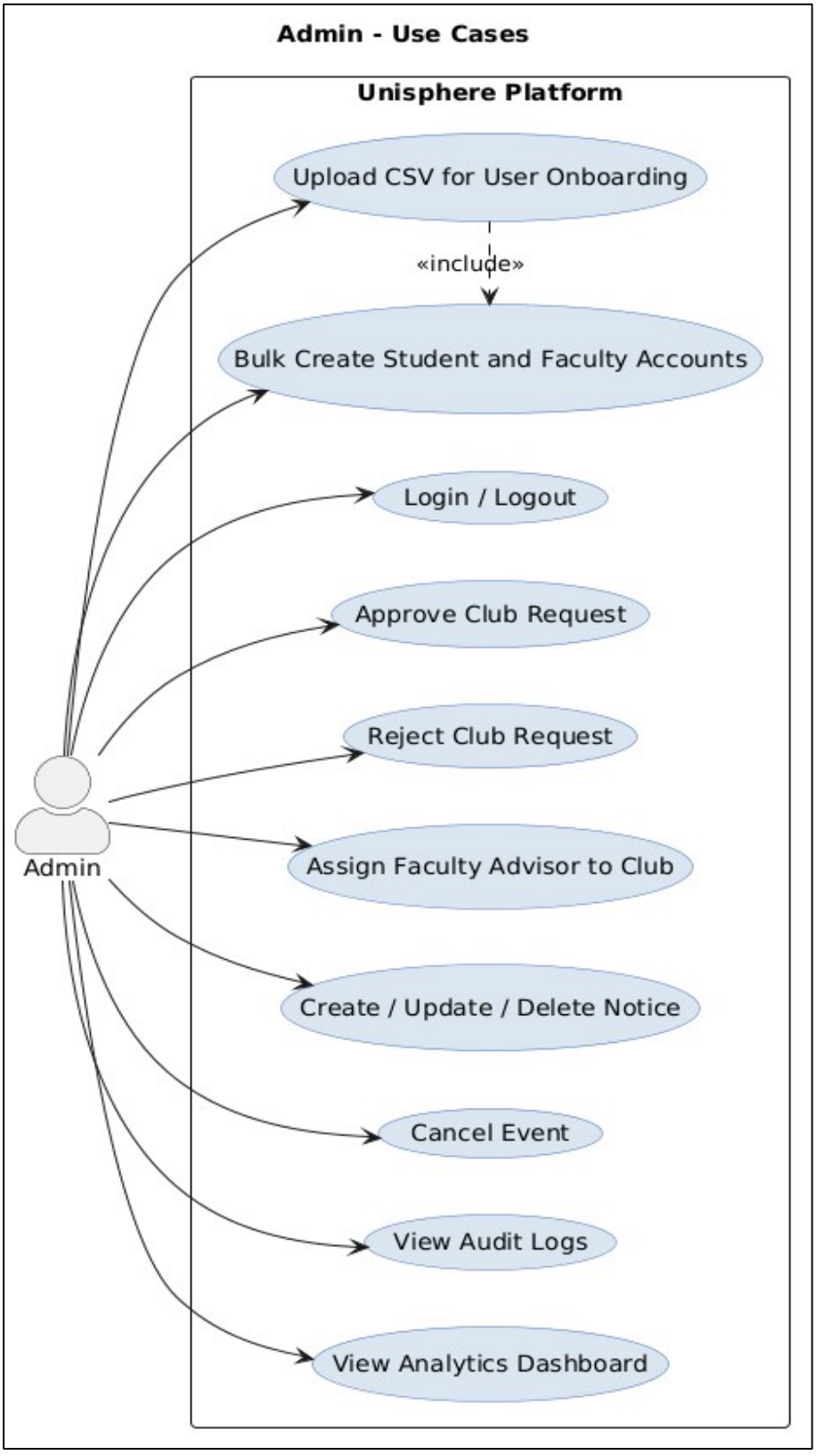
3.5. Faculty Use Case Diagram



The Faculty Use Case Diagram clearly specifies the responsibilities and tasks that are open to the faculty advisors on the platform. Faculty acts in the capacity of supervisors who evaluate the requests for events made by club coordinators. They can approve or reject these requests along with comments and also check the club activities they are supervising.

Moreover, faculty is informed about the activities carried out by the clubs through notifications. They can use messaging services provided on the platform. Approval process by the faculty is also depicted in this use case diagram. This ensures that all the student-led events pass through an evaluation before they become visible to everyone else on the platform.

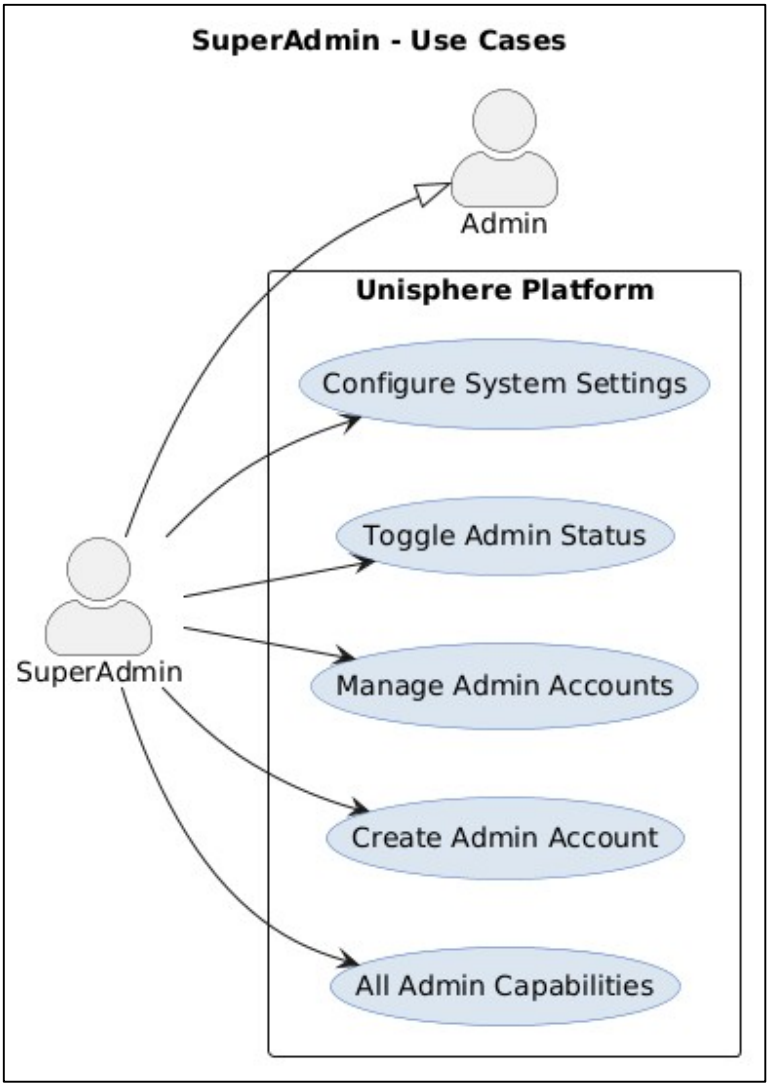
3.6. Admin Use Case Diagram



The Admin Use Case Diagram captures all the functionalities that administrators use in managing the platform at an institutional level. The administrator can perform user onboarding through CSV file upload where users' login credentials will be sent through emails.

The administrator can either accept or deny club formation requests, assign faculty members as advisers in clubs, manage notifications, cancel any events when necessary, and view activities performed in the entire platform using analytical reports. There is a feature to view audits of all the processes carried out including approval and denial of club formation requests among others.

3.7. SuperAdmin Use Case Diagram

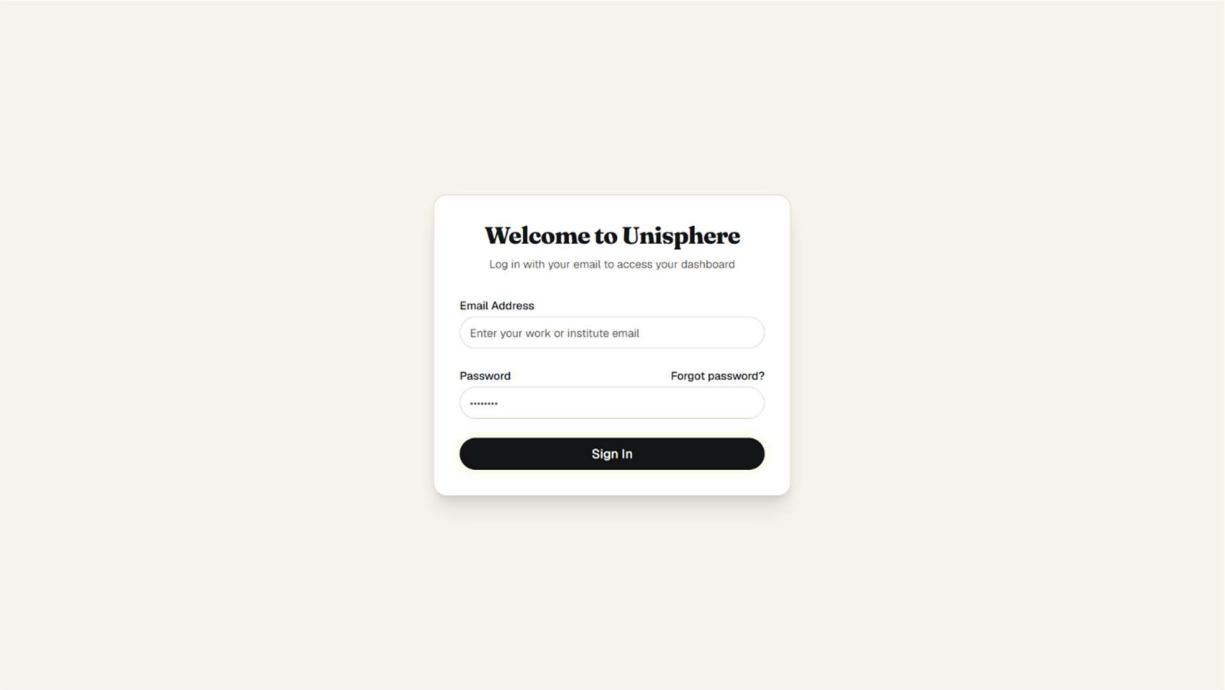


SuperAdmin Use Case Diagram shows the ultimate power that controls the Unisphere application. In addition to being endowed with all the powers an ordinary admin enjoys, the superadmin is equipped with the power to manage administrative accounts, modify the power of access of administrators, and manage the system's configurations using the system configuration module.

The superadmin power is meant to enable the management of the system on a level of organization so that the system can be effectively governed.

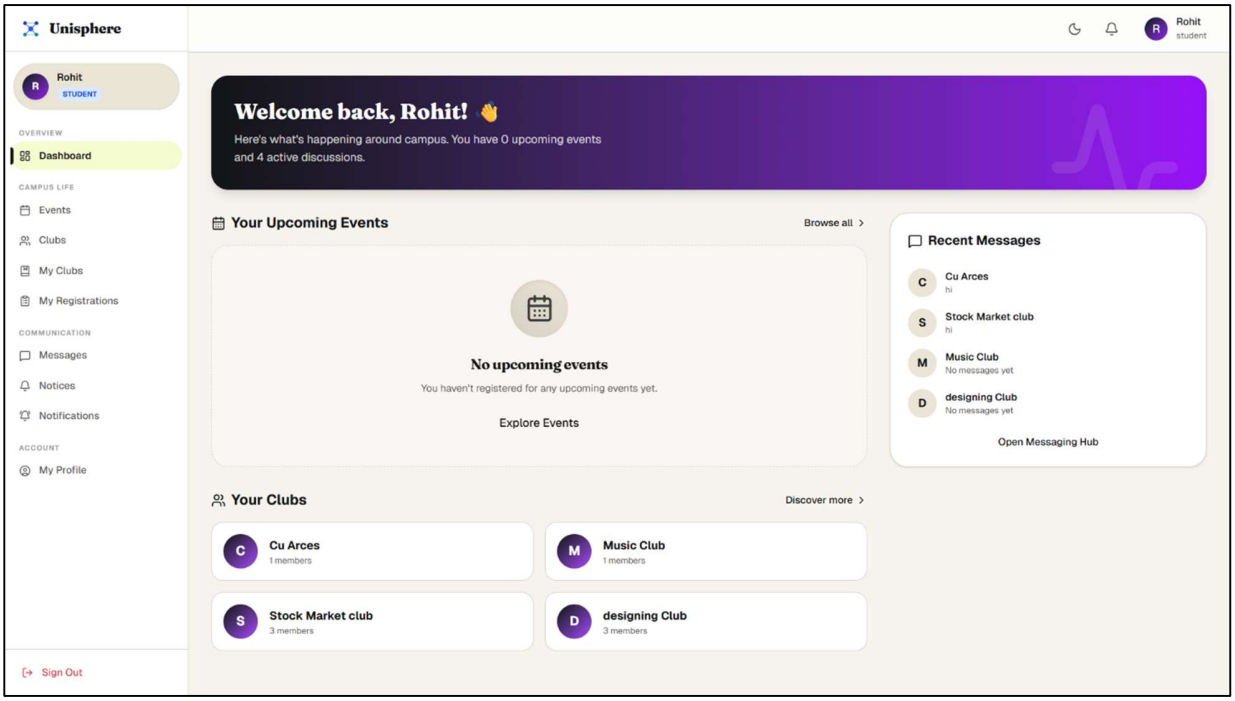
4. User Interface Design

4.1. Login Interface



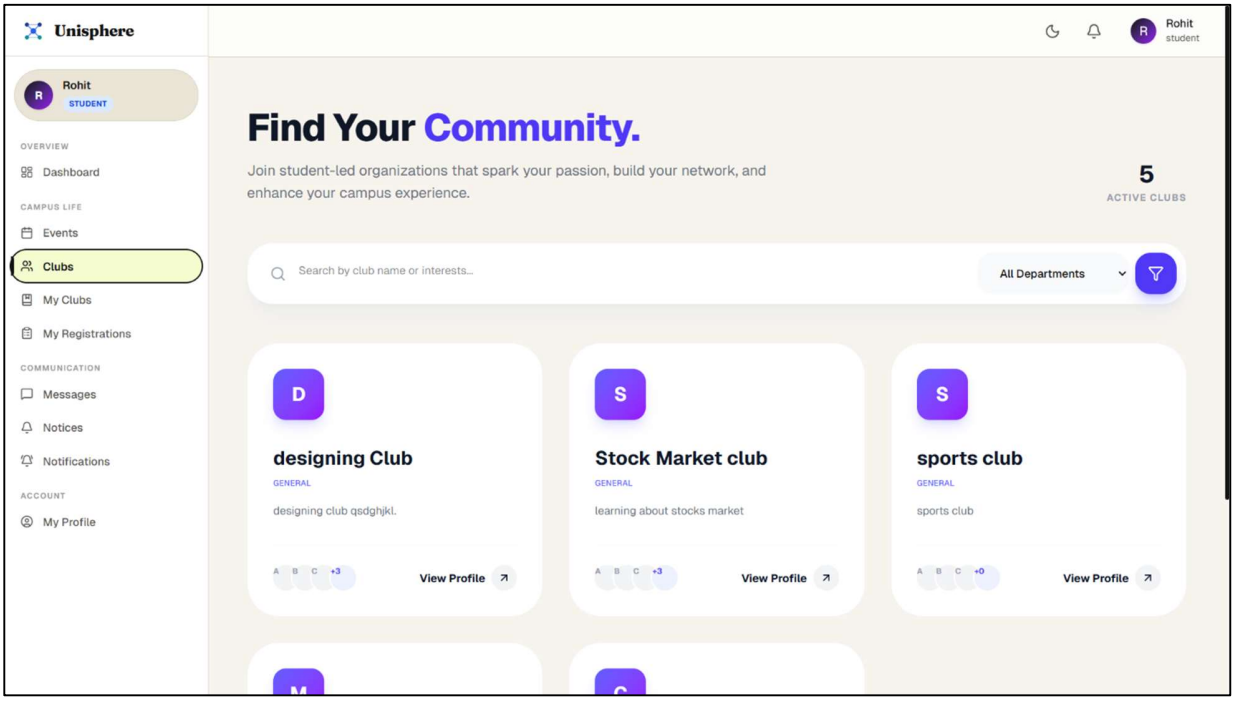
Secure role-based login interface for students, faculty, and administrators.

4.2. Student Dashboard



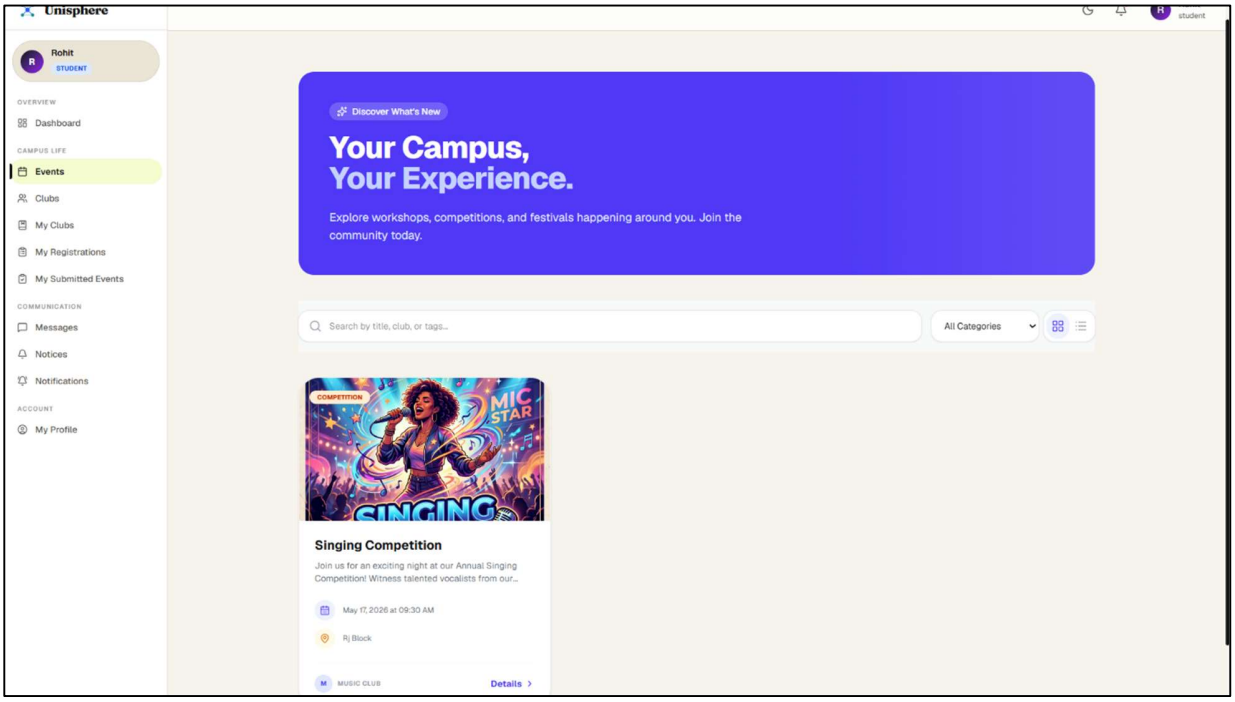
Student dashboard displaying upcoming events, joined clubs, notifications.

4.3. Club Discovery Interface



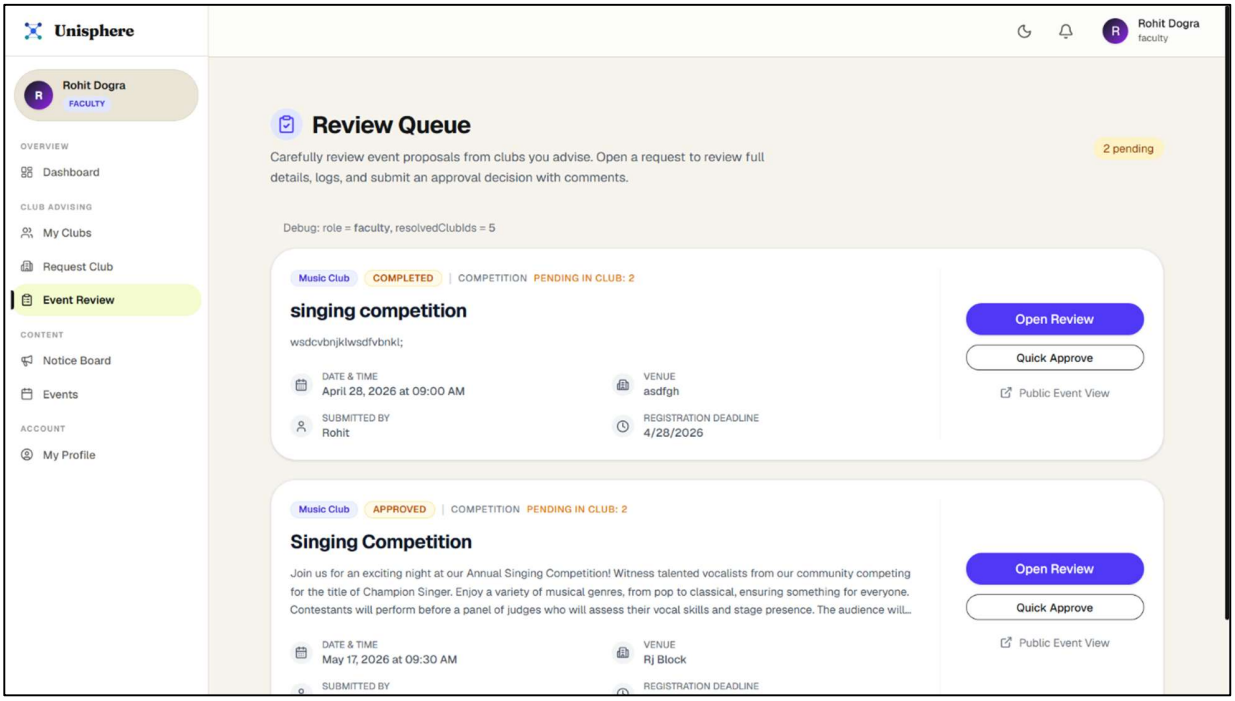
Interest-based club discovery interface allowing students to explore and join university clubs.

4.4. Event Discovery Interface



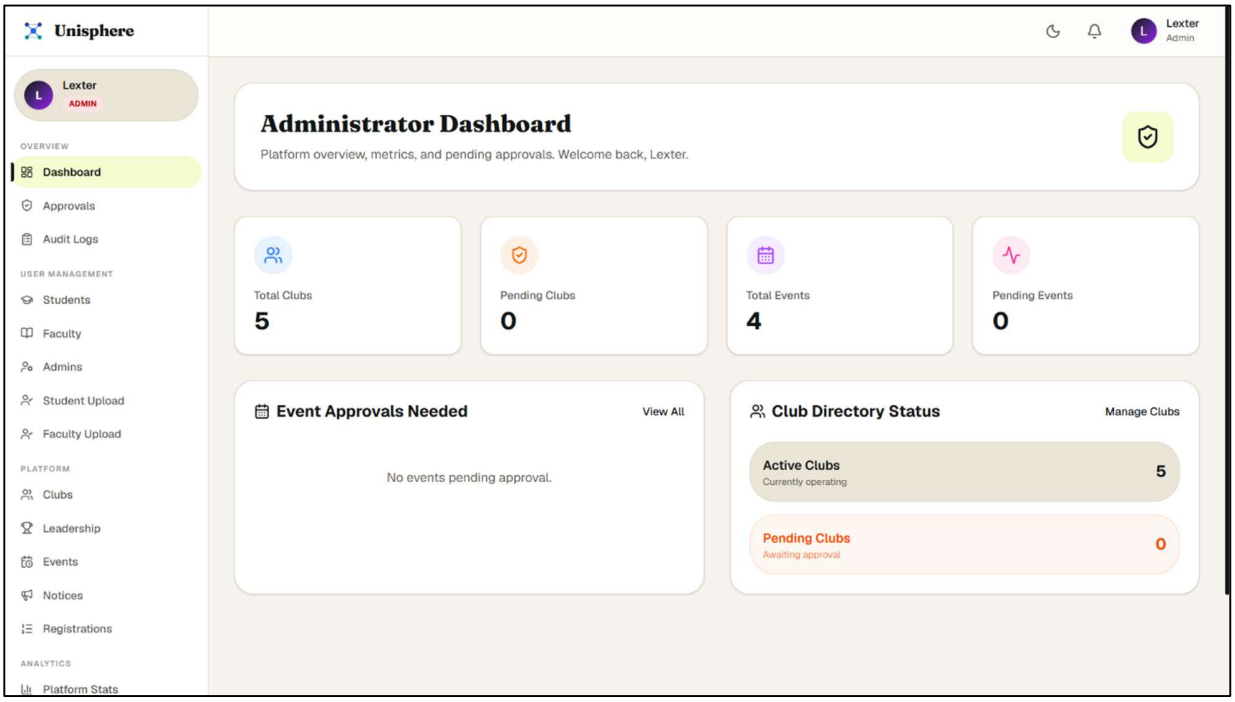
Event browsing interface showing approved university events with registration functionality.

4.5. Faculty Event Approval Interface



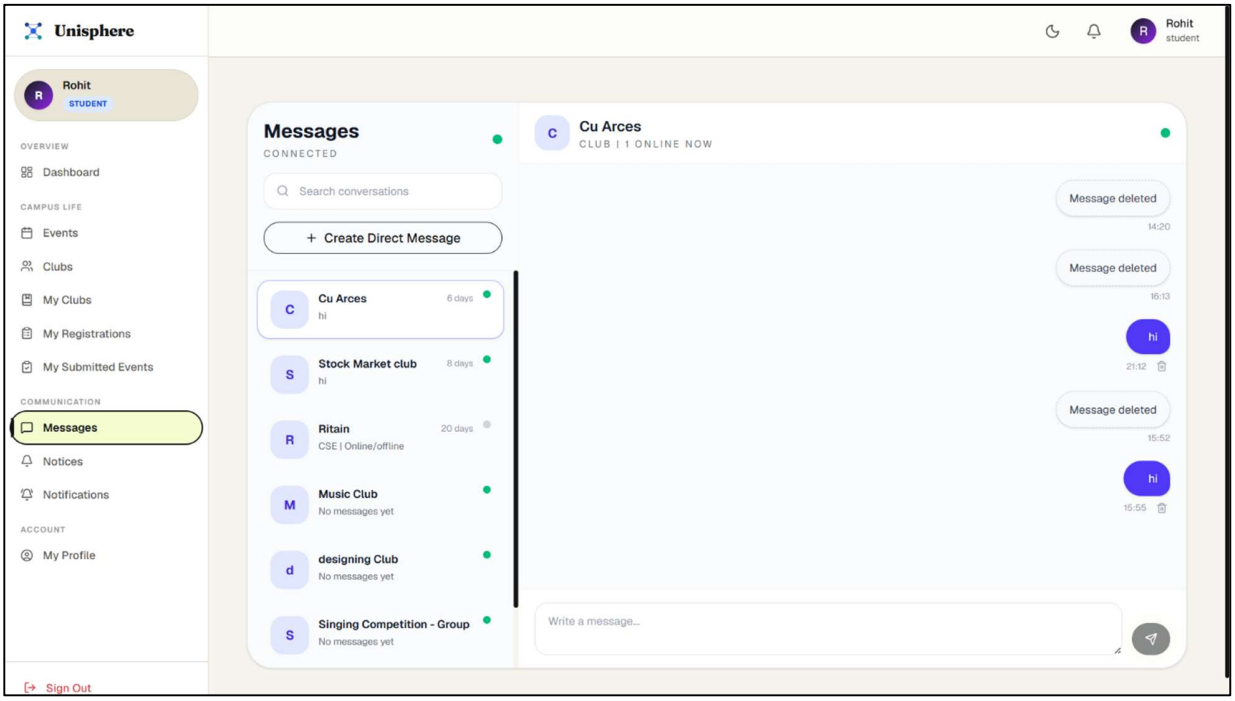
Faculty dashboard for reviewing, approving, or rejecting event requests submitted by clubs.

4.6. Admin Dashboard



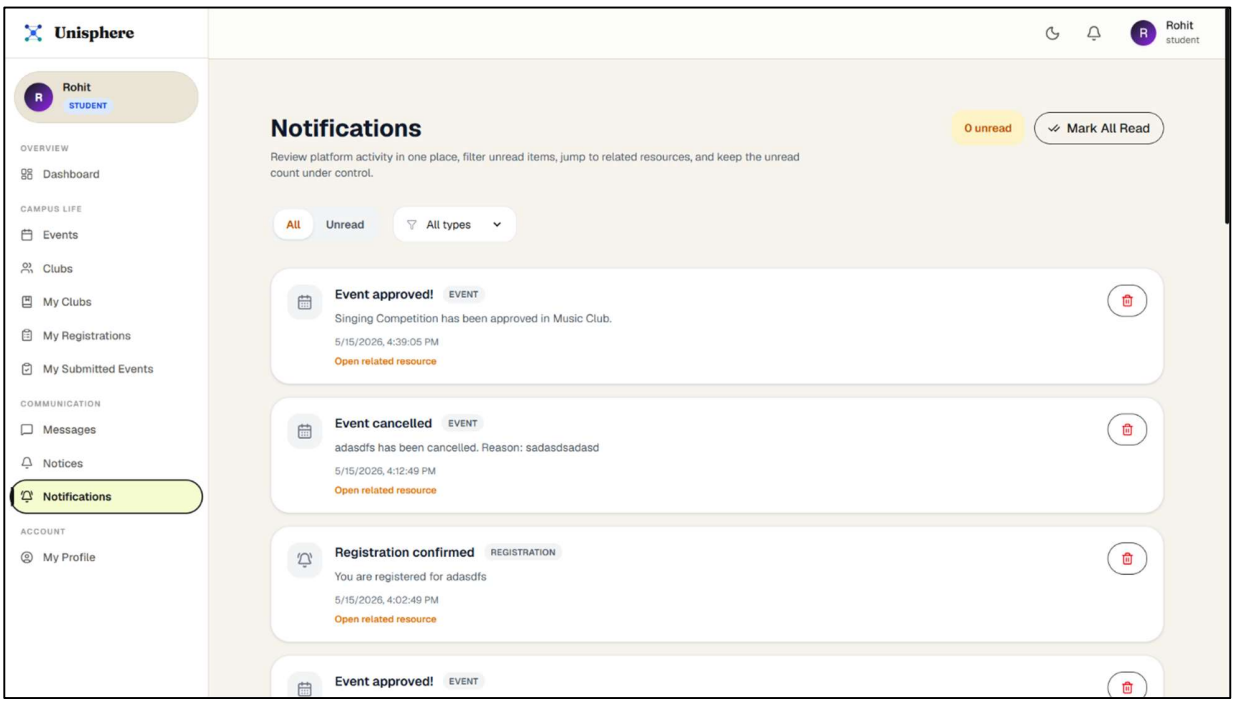
Administrative dashboard providing platform analytics, user management, and approval monitoring.

4.7. Messaging Interface



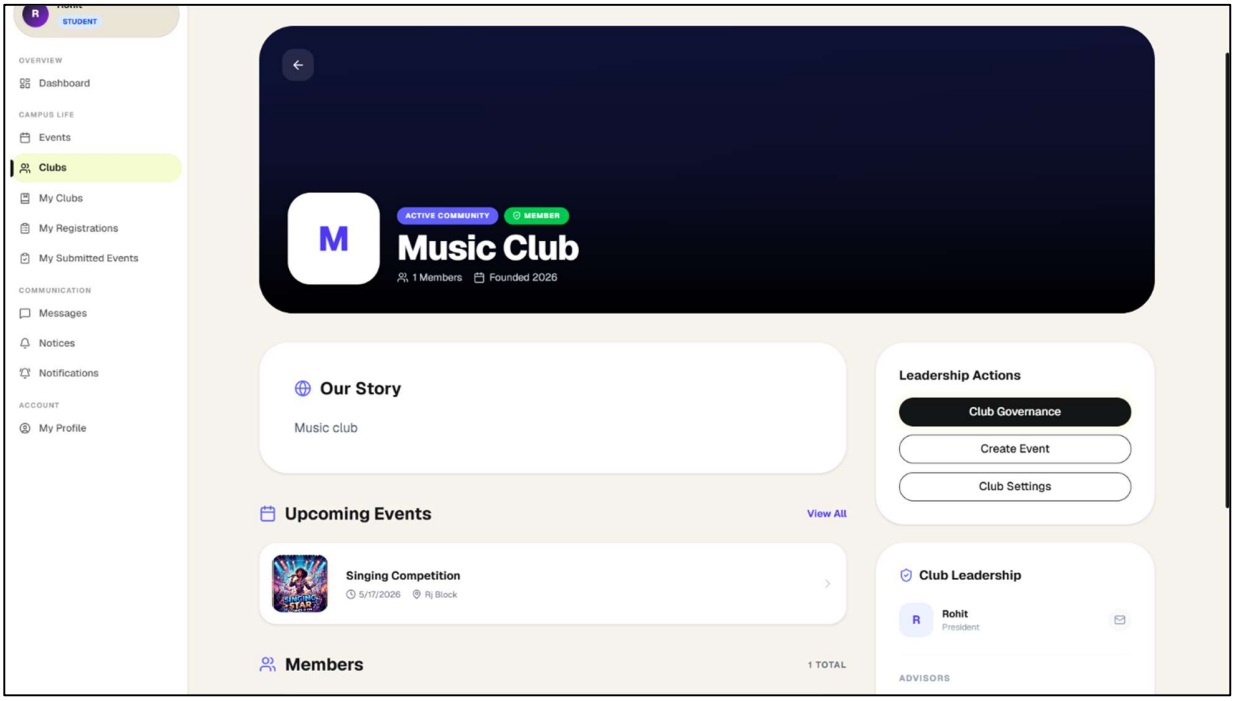
Real-time messaging interface supporting club chats, event groups, and direct conversations.

4.8. Notification Management Interface



Notification center displaying real-time updates related to events, registrations, and club activities.

4.9. Club Profile Interface



Club profile page displaying club details, leadership information, members, and upcoming events.

5. Tools and Technologies

5.1. Frontend Technologies

Technology	Purpose
React.js	Core frontend library for building component-based user interfaces
Vite	Build tool providing fast ES module development server and optimized production builds
React Router	Client-side routing with nested role-specific routes for student, faculty, and admin dashboards
Tailwind CSS	Utility-first CSS framework for responsive and consistent UI design
Framer Motion	Animation library for page transitions, micro-interactions, and UI motion effects
React Hook Form + Zod	Form state management and client-side schema validation
Axios	HTTP client for REST API communication with centralized error handling
TanStack React Query	Server-state management, API response caching, and background data synchronization
Socket.io-client	Real-time bidirectional communication for group chats, direct messages, and notifications
Recharts	Chart library used in the admin analytics dashboard for visualizing platform statistics

5.2. Backend Technologies

Technology	Purpose
Node.js	Server-side JavaScript runtime environment
Express.js	REST API framework for defining routes, middleware, and controllers
MongoDB Atlas	Cloud-hosted NoSQL document database for all platform data storage
Mongoose	ODM library for MongoDB schema definition, validation, and querying
JWT	Stateless authentication using access and refresh tokens stored in HTTP-only cookies
bcrypt	Password hashing with salt and configurable cost factor for secure credential storage
Socket.io	Real-time bidirectional communication for group chat, typing indicators, and presence
BullMQ	Background job queue management for scheduled and automated tasks
ioRedis	Redis client used by BullMQ as the underlying job queue store
node-cron	Cron job scheduler running every five minutes for automatic event status transitions
Nodemailer	Automated email delivery for sending onboarding credentials to users
csv-parser	CSV file parsing for bulk student and faculty import during onboarding

Cloudinary + Multer	Cloud-based file and image storage for profile pictures, event posters, and chat attachments
Firebase Admin	Push notification delivery to users via Firebase Cloud Messaging when FCM token is available

5.3. Development and Deployment Tools

Tool	Purpose
VS Code	Primary code editor used for development
Git + GitHub	Version control and source code repository management
Postman	API testing and documentation during backend development

6. Implementation

6.1. Project Structure

The project consists of two main folders: frontend and backend. Frontend is developed with React.js technology and is feature-based, including separate folders for components, pages, hooks, context providers, API services, and routing. Role-based dashboards for students, lecturers, and admins are included in protected routes. Backend consists of eighteen route modules with separate modules for each different feature. Each module contains a controller, a service file, and middleware based on the responsibility principle.

6.2. Authentication and Security

This is implemented using the stateless JWT model. On successful authentication, the server generates a short-lived access token with an expiration period of one day and a long-lived refresh token that expires after seven days, both of which are saved in HTTP-only cookies. The protected routes are then secured by two middlewares: the verifyJWT middleware, which checks the validity of the token, and the verifyRole middleware, which makes sure the token belongs to a user with the right permissions for accessing the specific route. The passwords are encrypted with bcrypt before being saved. Other security features include rate-limiting to 1000 requests within 15 minutes and setting up CORS to be restricted to the frontend's URL only.

6.3. User Onboarding

The administrator will be able to upload a CSV file containing the records of either students or faculty through the admin panel. In the backend system, the uploaded CSV file will be parsed with the help of the csv-parser library, creating user profiles in the database, generating a one-time password for each user, and sending the credentials to the user's registered email through Nodemailer. Once done, the users will be able to log in using the received credentials and set up a password.

6.4. Club and Event Management

Club creation begins when a student becomes the club president following approval of the club. The administration appoints a faculty advisor before the club becomes active. The students find

clubs using the club directory, which makes suggestions based on interest tags chosen during the onboarding process.

Event creation is carried out by club administrators and runs a life cycle consisting of six stages: Draft, Pending, Approved, Live, Completed, and Cancelled. Every five minutes, a node-cron job is run to change the status of events based on their scheduled start and end times. Once the event has been approved, an automatic creation of the EventGroup chat room happens for all the members. Once the event goes into the Completed status, a BullMQ job removes the group chat after 24 hours using up to three retries.

6.5. Real-Time Communication

Socket.io is the technology that powers all real-time aspects of the project. The application offers three types of chat rooms: EventGroup, which is used by event participants; Club, used by club members; and DirectConversation, used to provide one-on-one communication between two users. Typing indicators and user presence detection are managed using socket-specific events. Soft deletion, which makes messages invisible but keeps records of them in the database, is possible.

6.6. Notification System

There are two methods of notifications. First, there is an in-app notification that is saved in the notifications collection and communicated in real-time using Socket.io. The other method is push notification, which utilizes Firebase Cloud Messaging, provided the user has an FCM token. Notifications include any significant event on the platform, such as approving events, rejecting them, registering events, and club management activities.

6.7. Administrative Dashboard

Visibility of the whole system is provided using the admin dashboard. The analytics page employs Recharts to present charts on the number of users according to their roles, active clubs, status of events, and registrations over time. The admin can check pending applications for clubs and grant permission, assign faculty members as advisors, and check the audit log collection containing approvals and rejections with reasons. Event registration information can be downloaded in CSV format.

7. Major Findings/Outcomes/Output/Results

7.1. Key Outcomes

Unisphere has brought about the establishment of a complete university platform that efficiently resolves all the three major issues that the system aimed to solve:

- Fragmented club discovery, poor event awareness, and the lack of proper institution oversight.

The following results have been realized:

- A central platform was created whereby students, faculty members, and administrators communicate through role-oriented dashboards with no overlapping capabilities.
- The club discovery feature works by curating a list of clubs for students according to their interest tags entered during onboarding.
- Event approval enables the system to ensure that no event is made available to students without the consent of the faculty advisor.
- The CSV onboarding method allows for no self-registration processes and guarantees that only legitimate members of the institution can use the platform.
- In real-time communications are facilitated through Socket.io, which enables group chats for each event, group chats for each club, and one-on-one messaging among users.
- The event lifecycle management capability enables event statuses to be automatically updated according to schedule using node-cron.
- BullMQ ensures automatic dissolution of event groups within 24 hours after events have taken place.
- An audit log is kept in the approvallogs database for every approval and rejection decision made along with faculty comments.

7.2. Platform Features Delivered

Feature	Status
CSV-based bulk user onboarding with automated credential emails	Implemented
Interest tag-based club discovery and recommendation	Implemented
Six-stage event approval workflow with faculty gating	Implemented
Real-time group chat for clubs and events	Implemented
One-to-one direct messaging between users	Implemented
In-app and push notification system with read/unread management	Implemented
Automatic event status transitions based on scheduled timing	Implemented
Automatic event group dissolution after event completion	Implemented
Admin analytics dashboard with platform-wide statistics	Implemented
Full audit log for all approval and rejection actions	Implemented
Post-event feedback collection with ratings and comments	Implemented
Role-based access control with secure JWT authentication	Implemented
Official notice board for faculty and admin announcements	Implemented

7.3.Technical Findings

- The caching and invalidation capabilities of React Query ensured that the frontend remained consistent with the backend while avoiding unnecessary API requests.
- BullMQ's retry mechanism with an exponential backoff strategy using three tries solved Redis connectivity problems automatically without any manual effort needed.
- The clear separation of roles in the middleware layer and schema ensured easy extension; adding a role or permission involved modifications in just one place.

8. Conclusion and Future Scope

8.1. Conclusion

Unisphere has been created to fill an existing need in nearly every college and university but one that often goes unaddressed. College students miss out on clubs, activities, and opportunities not because they are not interested but because there is no infrastructure for connecting with these possibilities. There are no systems for managing club events by faculty members to ensure accountability, structure, and maintainability and an ability to see what decisions have been made in previous years.

This solution offers solutions for all three of these issues. It provides a well-defined entry point for students to connect with university clubs based on their interests. It enables professors to control which clubs they advise and manage what events go live in the application. It allows administrators full transparency over the process and access to every decision made, along with a way to onboard users using CSV files and track activity.

The development process proves that even a professional-level full-stack application using MERN Stack with real-time communications, jobs management, cloud storage, push notifications, and authentication can be developed and deployed by one developer only. Every major functionality was tested thoroughly and works properly.

8.2. Future Scope

- **Mobile Application** — The use of an iOS and Android app developed with React Native will enable students to connect with the platform and get alerts from their phone.
- **QR Code Attendance** — Each event can have one-time limited-use QR codes that students can use to check in at the event venues without proxy attendance.
- **AI-Powered Club Recommendations** — An algorithm that analyzes student activities and behaviors will generate recommendations of clubs beyond just matching interests.
- **Calendar Integration** — Approved events can be synced with Google Calendar and Outlook so that students get notifications via the existing calendar app.
- **Analytics for Club Admins** — Club presidents will be able to analyze data about membership numbers, event participation, and other relevant metrics on dashboards.

9. References

[1] Meta Open Source, "React – A JavaScript Library for Building User Interfaces," React Documentation. [Online]. Available: https://react.dev/ [Accessed: May 2026].

[2] OpenJS Foundation, "Node.js Documentation." [Online]. Available: https://nodejs.org/ [Accessed: May 2026].

[3] OpenJS Foundation, "Express.js – Fast, Unopinionated, Minimalist Web Framework for Node.js." [Online]. Available: https://expressjs.com/ [Accessed: May 2026].

[4] MongoDB, Inc., "MongoDB Atlas Documentation." [Online]. Available: https://www.mongodb.com/atlas [Accessed: May 2026].

[5] Socket.IO Contributors, "Socket.IO Documentation – Real-Time Bidirectional Event-Based Communication." [Online]. Available: https://socket.io/ [Accessed: May 2026].

[6] Auth0, "Introduction to JSON Web Tokens." [Online]. Available: https://jwt.io/introduction [Accessed: May 2026].

[7] Taskforce.sh, "BullMQ – Premium Message Queue for Node.js." [Online]. Available: https://docs.bullmq.io/ [Accessed: May 2026].

[8] Redis Ltd., "Redis Documentation." [Online]. Available: https://redis.io/docs/ [Accessed: May 2026].

[9] Google, "Firebase Cloud Messaging – Documentation." [Online]. Available: https://firebase.google.com/docs/cloud-messaging [Accessed: May 2026].

[10] Nodemailer Contributors, "Nodemailer – Send Emails from Node.js." [Online]. Available: https://nodemailer.com/ [Accessed: May 2026].

[11] Cloudinary Ltd., "Cloudinary Documentation." [Online]. Available: https://cloudinary.com/documentation [Accessed: May 2026].

[12] Tailwind Labs, "Tailwind CSS – A Utility-First CSS Framework." [Online]. Available: https://tailwindcss.com/ [Accessed: May 2026].